SHA (Secure Hash Algorithm)

1. Overview

SHA refers to a family of **cryptographic hash functions** designed by the NSA and standardized by NIST.

A hash function takes an input (message) of any length and produces a **fixed-length, deterministic digest**. Hashing is a one-way operation used for verification and integrity, never for confidentiality (a hash cannot be "decrypted" back to its input).

SHA at a Glance

Property	Detail
Type	Cryptographic hash function family
Output	Fixed-length digest (varies by variant)
Reversibility	One-way — computationally infeasible to reverse
Determinism	Same input always produces the same output

---

2. Required Properties of a Secure Hash Function

A secure cryptographic hash function should provide:

1. **Pre-image resistance** — given a hash output `h`, it should be infeasible to find any input `m` such that `hash(m) = h`.
2. **Second pre-image resistance** — given an input `m1`, it should be infeasible to find a different input `m2` such that `hash(m1) = hash(m2)`.
3. **Collision resistance** — it should be infeasible to find *any* two distinct inputs `m1` and `m2` such that `hash(m1) = hash(m2)`.

---

3. The SHA Family

Variant	Digest Size	Status
SHA-1	160 bits	**Deprecated and broken** — a practical collision was publicly demonstrated in 2017; must not be used for security purposes.

SHA-2 (SHA-256)	256 bits	Secure, widely adopted current standard.
SHA-2 (SHA-512)	512 bits	Secure, used where a larger digest or 64-bit-optimized performance is desired.
SHA-2 (SHA-384, SHA-224)	384 / 224 bits	Truncated SHA-512/SHA-256 variants for specific length requirements.
SHA-3 (Keccak-based)	Variable (224–512 bits)	A structurally different design (sponge construction) standardized as a hedge against any future structural weaknesses discovered in SHA-2.

SHA-2 vs. SHA-3 Structure

SHA-2 uses a **Merkle–Damgård construction**, processing the message in fixed-size blocks through a compression function.

SHA-3 instead uses a fundamentally different **sponge construction**, deliberately designed to be structurally independent of SHA-2 so that a break in one family would not automatically compromise the other.

---

4. Why SHA-1 Was Broken and MD5 Is Worse

SHA-1's 160-bit output and internal design left it vulnerable to collision attacks — finding two different inputs producing the same hash — once sufficient computational resources became available.

A costly but ultimately successful attack was publicly demonstrated in the **"SHAttered" attack (2017)**.

SHA-2's larger digest sizes and revised internal structure have, to date, resisted any comparable practical break.

---

5. What Hashing Is and Isn't Good For

Appropriate Uses

- **File and data integrity verification** — confirming a download or transfer wasn't corrupted or tampered with.
- **Digital signature construction** — signing a hash of a message rather than the message itself, for efficiency.
- **Blockchain transaction and block linking.**

- **Deduplication and content-addressable storage.**

Inappropriate Uses

- **Password storage** — SHA-256/SHA-512 are *fast* hash functions, meaning an attacker with a dumped hash can attempt billions of guesses per second on modern hardware.
- Fast general-purpose hashes were never designed to resist this kind of brute-force guessing attack against a low-entropy secret like a human password.
- This is precisely why dedicated password-hashing functions (**bcrypt, Argon2, PBKDF2**) exist and must be used instead (see their respective guides).

> **Key point:** General-purpose SHA hashes are appropriate for integrity and authentication-related uses, but they are the wrong tool for password storage.

---

6. Salting and Hashing

A **salt** is a unique, random value combined with the input before hashing.

Salting ensures that identical inputs (for example, two users with the same password) produce different hash outputs, defeating precomputed lookup-table attacks such as **rainbow tables**.

Why Salting Alone Is Not Enough

Salting alone does **not** make a fast hash function like SHA-256 suitable for password storage.

It only removes the precomputation shortcut. The fundamental speed problem — potentially billions of guesses per second — remains unaddressed.

Therefore:

> **Salting is necessary but not sufficient. Dedicated slow/memory-hard password-hashing functions remain the correct choice for passwords specifically.**

---

7. HMAC — Keyed Hashing for Authentication

SHA functions are frequently combined with a secret key via **HMAC (Hash-based Message Authentication Code)** to provide message authentication.

HMAC verifies:

- **Message integrity**
- That the message originated from someone possessing the **shared secret key**

This provides authentication in addition to integrity, rather than integrity alone.

Simplified HMAC Concept

```
Message + Shared Secret Key
            |
            ▼
           HMAC
            |
            ▼
    Authentication Tag
```

---

8. Typical Use Cases

SHA-family functions are commonly used for:

- **TLS/SSL certificate fingerprints and integrity checks**
- **Git commit hashing** — SHA-1 historically, transitioning to SHA-256
- **Code signing and software integrity verification**
- **HMAC-based API authentication**
- **Blockchain proof-of-work and block hashing** — e.g., Bitcoin uses double SHA-256

---

9. Summary

SHA is a family of general-purpose cryptographic hash functions essential for **data integrity, digital signatures, and authentication**.

However, its member functions are deliberately **fast**, which makes them the wrong tool for password storage specifically.

> **SHA-1 is broken and must be avoided. SHA-256/SHA-512 and SHA-3 remain secure and are the current standard for integrity and signature use cases.**

Final Takeaway

```
SHA Family
    │
    ├── Integrity
    ├── Digital Signatures
    ├── Authentication / HMAC
    └── Blockchain / Data Verification

NOT for:
    └── Password Storage
         ↓
Use dedicated password-hashing functions
such as bcrypt, Argon2, or PBKDF2
```